

1. Los periféricos del procesador

Un procesador, o la idea de procesador que podemos tener en general (máquina que ejecuta instrucciones secuencialmente) es sólo una pequeña parte de toda la historia. La parte ejecutora de instrucciones es el núcleo o *core*. Con esto, el procesador sólo sería capaz de procesar instrucciones... y poco más.

Para delegar funciones, y permitir interconectarlo con el exterior, un procesador se rodea de periféricos. Estos periféricos se pueden configurar para realizar, de manera autónoma, funciones que de otra manera tendrían ocupado al procesador. Algunos de los más comunes son temporizadores, manejadores de protocolos de intercomunicación, aceleradores hardware para funciones complejas (seguridad, criptografía...), motores de acceso a memoria directa (DMAs), etc.

Todos estos periféricos se manejan desde direcciones de memoria, donde se mapean todos los registros internos. En nuestro caso concreto, y como ejemplo, el procesador **STM32F723IE** cuenta con un bus principal de memoria de tipo AXI (*Advanced Extensible Interface*) para enviar y recibir datos del espacio de memoria. Numerosos buses secundarios de tipo AHB (*Advanced High-Performance Bus*) se conectan a través de una matriz de interconexión ([rm0431 2](#)), que a su vez sirven diferentes subconjuntos de dispositivos como se muestra en la Figura 1.

Todas estas interconexiones entre buses se identifican, a nivel de hardware, mediante direcciones. En cada punto, y según la dirección de memoria solicitada, se establecerá una conexión entre el origen y destino de la transferencia de datos. Esto es automático desde el punto de vista del procesador o periféricos, que únicamente tendrán que colocar sus datos (o peticiones) y esperar a la confirmación de transacción exitosa.

1.1. El bus AXI

El bus *AXI* (*Advanced Extensible Interface*) es el medio que tiene el procesador de enviar sus peticiones al exterior. Perteneciente a la especificación AMBA (*Advanced Microcontroller Bus Architecture*) de ARM, es una interfaz de bus con gran ancho de banda y frecuencia, y baja latencia.

AXI se basa en un esquema de cinco canales independientes para la transferencia de datos y direcciones:

- **Canal de dirección de lectura (*Read Address*):** Especifica la dirección de memoria para lectura y señales de control.
- **Canal de datos de lectura (*Read Data*):** Transporta los datos leídos desde el esclavo hacia el maestro, junto con la respuesta de estado (*OKAY*, *ERROR*, etc.).
- **Canal de dirección de escritura (*Write Address*):** Especifica la dirección de memoria para escritura y señales de control.
- **Canal de datos de escritura (*Write Data*):** Transporta los datos a escribir.
- **Canal de respuesta de escritura (*Write Response*):** Permite notificar al maestro del resultado final de la operación de escritura.

El dividir el bus en canales de subida y bajada permite lecturas y escrituras simultáneas. Adicionalmente, las señales internas permiten transferencias en ráfaga o incluso fuera de orden.

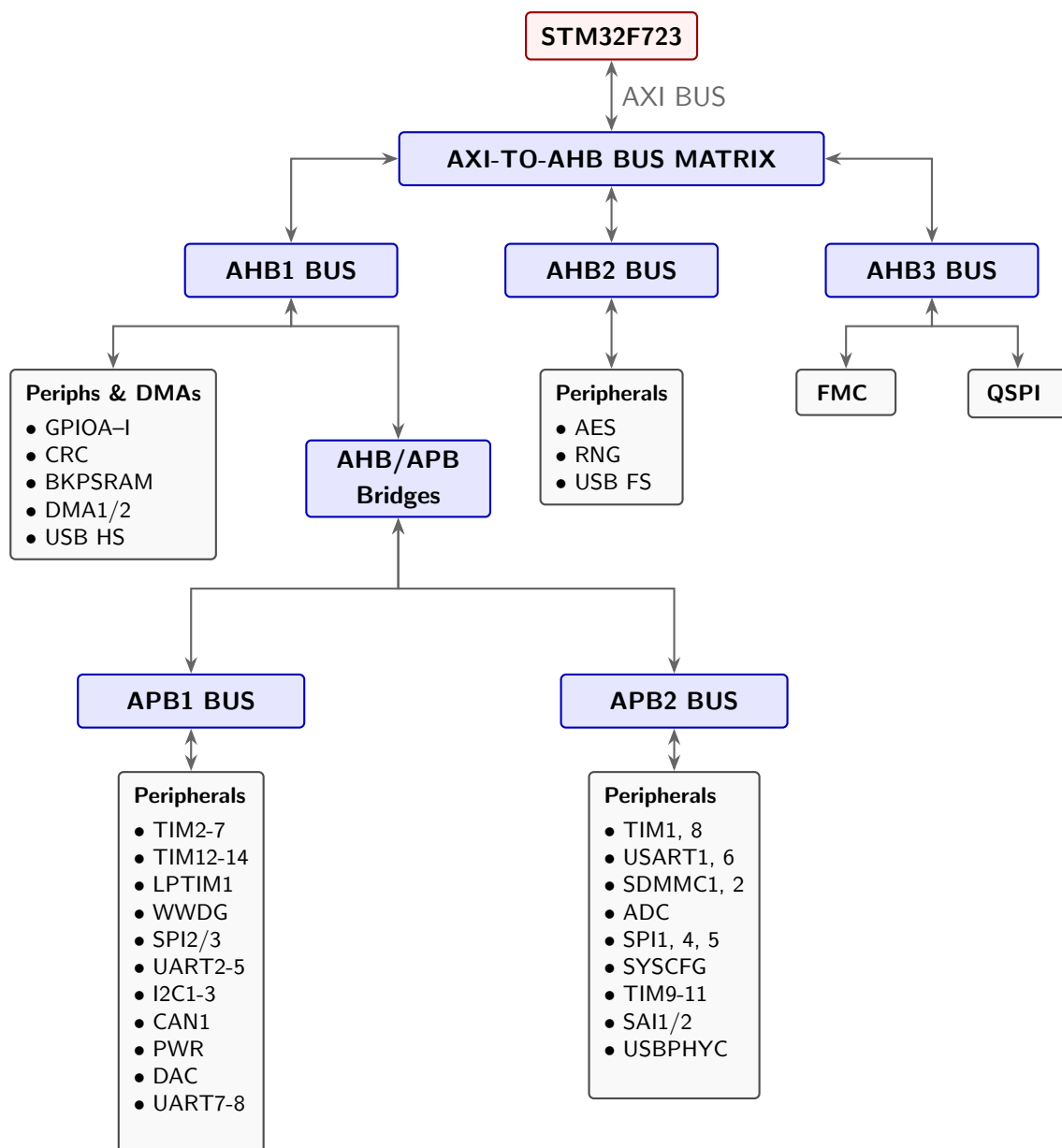


Figura 1: Interconexión del procesador y dispositivos

1.1.1. *Handshake* o sincronización

Todas las transferencias en cada uno de los 5 canales de AXI utilizan un mecanismo uniforme de sincronización denominado *handshake* bidireccional mediante dos señales principales:

- **VALID:** Generada por el emisor (origen) para indicar que las direcciones, datos o señales de control presentes en el canal son válidos.
- **READY:** Generada por el receptor (destino) para indicar que está listo para aceptar la información.

La transferencia de datos ocurre exclusivamente en el flanco de reloj ascendente en el que **ambas** señales (**VALID** y **READY**) se encuentran activas en nivel alto (1), tal como muestra la Figura 2.

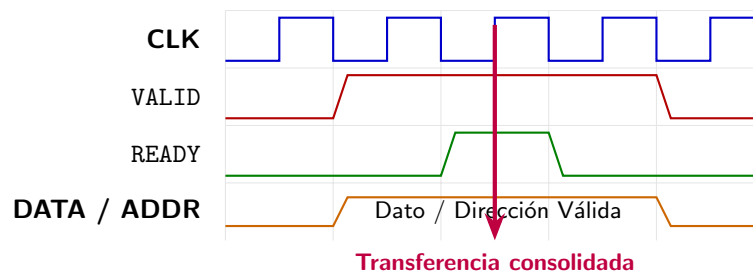


Figura 2: Sincronización por *handshake* en un canal AXI. La transferencia se realiza únicamente cuando **VALID** y **READY** coinciden en nivel alto durante el flanco del reloj.

1.2. El bus AHB y APB

El bus *AHB* interconecta el procesador con las memorias internas (SRAM, Flash) y los periféricos (como los controladores DMA, Ethernet o USB). Aunque se podría utilizar AXI, AHB es más sencillo y evita una compleja lógica de control en cada uno de los periféricos, mientras que mantiene un rendimiento elevado.

Su funcionamiento es mucho más sencillo, con dos fases (envío de dirección y envío de datos) que se suceden en el tiempo. Para mejorar el rendimiento, estas fases se pueden solapar formando un pipeline.

Los periféricos sencillos o de baja velocidad no se conectan directamente al bus de altas prestaciones (AHB) debido a sus menores requerimientos de velocidad y a la necesidad de no recargar el bus principal. Para ello se emplean buses de periferia más simples, como el bus *APB* (*Advanced Peripheral Bus*), y elementos de adaptación denominados **puentes** (*bridges*).

1.3. El puente entre bus y registro

Un puente (*Bridge*) actúa como un esclavo dentro del bus de alta velocidad (AHB) y como el único maestro en el bus periférico (APB). Su función principal es traducir las transacciones de alta velocidad de AHB en lecturas y escrituras de 2 ciclos de reloj (dirección y dato) en APB.

Cuando se realiza una lectura o escritura en una dirección de memoria correspondiente a un periférico, se sigue el siguiente flujo:

1. **Decodificación en bus/puente:** El decodificador examina los bits superiores de la dirección del bus hasta identificar el periférico correspondiente.
2. **Selección de Periférico:** El puente activa la señal de selección individual correspondiente al periférico deseado (PSELx).
3. **Offset de Registro:** Los bits inferiores de la dirección seleccionan el registro interno dentro del periférico.
4. **Escritura / Lectura:** El periférico lee o escribe en el registro indicado los datos indicados.

1.4. La interconexión con diferentes memorias

La interconexión con las memorias se hace de manera similar a como se hace con el resto de los periféricos. Al fin y al cabo, recibirán una dirección y dato, además de una operación (escritura/lectura) y con tan solo esa información podrán operar. Sin embargo, además de los buses habituales, suelen existir otros buses secundarios que ofrecen accesos directos.

1.4.1. Memorias acopladas mediante ITCM y DTCM

Para evitar saturar la matriz de buses cuando el núcleo ejecuta instrucciones a frecuencias muy elevadas, el procesador suele integrar interfaces de memorias con cero estados de espera (*Zero Wait State*).

- **ITCM (*Instruction Tightly-Coupled Memory*):** Bus y memoria dedicados exclusivamente a la búsqueda y ejecución de código.
- **DTCM (*Data Tightly-Coupled Memory*):** Bus y memoria dedicados a datos críticos de tiempo real.

1.4.2. Controladores de memoria externa (FMC, QSPI, Flash...)

Si los requisitos del sistema piden más memoria que la alcanzable con las anteriores interfaces, se utilizan periféricos, ya a través de los buses AHB, que interconectan con memorias externas de mayores tamaños. Por ejemplo:

- **FMC (*Flexible Memory Controller*):** Permite conectar memorias externas paralelas como SDRAM, SRAM o NOR Flash.
- **QSPI (*Quad-SPI Interface*):** Interfaz serie de alta velocidad para memorias Flash externas.

1.5. Ejemplos de periféricos

Los periféricos integrados en el microcontrolador abstraen al procesador de tareas de bajo nivel. Por ejemplo, para interconectar mediante dispositivos externos con protocolos como **USB** [rm0431 32,33](#), **UART** [rm0431 27](#), **SPI** [rm0431 28](#) ... en lugar de tener al procesador ocupado, tenemos periféricos que autogestionan la conexión y avisan al procesador sólo cuando hay una decisión que tomar.

También la gestión de entrada/salida mediante pines de propósito general es controlada mediante los periféricos **GPIO** [rm0431 6](#), por ejemplo para encender LEDs o leer el estado de botones.

O todo lo referente a temporización del sistema, mediante periféricos como los temporizadores **TIM** [rm0431 21](#) o el reloj de tiempo real **RTC** [rm0431 25](#).

1.6. Activación de periféricos

Un aspecto muy a tener en cuenta en microcontroladores modernos es la gestión de energía. Por defecto, todos los periféricos del microcontrolador están desactivados tras el reseteo (sus señales de reloj están desconectadas).

Para conectarlas y dar servicio a los periféricos, se deben activar uno a uno mediante señales específicas. Normalmente, un periférico es el responsable de esta activación, y en concreto en el **STM32F723I** es el **RCC** [rm0431 5](#).

1.6.1. El periférico **RCC** (*Reset and Clock Control*)

El periférico **RCC** [rm0431 5](#) gestiona los relojes del sistema (origen, frecuencia, forma...) y la habilitación individual del reloj de cada periférico.

Antes de poder leer o escribir en cualquier registro de un periférico, habrá que haber activado previamente su señal de reloj en el registro RCC correspondiente.